

MODULE 2

BASICS OF DIGITAL AUDIO

Basics of Digital Audio—Digitization of Sound, Musical Instrument Digital Interface (MIDI), Lossless Compression Algorithms— Introduction, Basics of Information Theory, Run-Length Coding, Variable-Length Coding, Dictionary-Based Coding, Arithmetic Coding, Lossless Image Compression.

DIGITIZATION OF SOUND

What is Sound?

Sound is a mechanical wave that propagates through a medium such as air, water, or solid objects. It is produced by the vibration of objects, which creates pressure variations in the surrounding medium. These variations are perceived by the human ear as sound.

- **Frequency (Pitch):** Measured in Hertz (Hz), frequency determines how high or low a sound is. Higher frequencies correspond to higher-pitched sounds.
- **Amplitude (Loudness):** The amplitude of a sound wave determines its loudness. Greater amplitude means a louder sound, while lower amplitude means a softer sound.
- **Timbre (Quality):** Timbre refers to the characteristics of a sound that distinguish it from others, even if they have the same pitch and loudness (e.g., the difference between a piano and a violin playing the same note).

Digitization of Sound

Digitization is the process of converting an analog audio signal into a digital format that can be stored and processed by computers. This process involves **sampling, quantization, and encoding**.

Steps in Digitization:

1. **Sampling:** The analog signal is measured at regular intervals to create discrete data points.
2. **Quantization:** The sampled values are approximated to the nearest discrete level.
3. **Encoding:** The quantized values are represented as binary numbers for storage and processing.

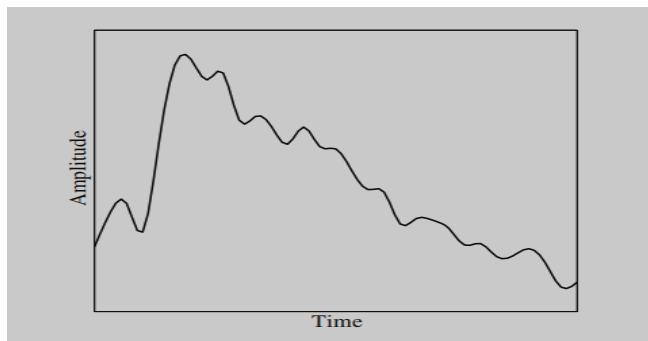


fig 6.1

Figure 6.1 shows the one-dimensional nature of sound. Values change over time in *amplitude*: the pressure increases or decreases with time [1]. The amplitude value is a continuous quantity. Since we are interested in working with such data in computer storage, we must *digitize* the *analog signals* (i.e., continuous-valued voltages) produced by microphones. For image data, we must likewise digitize the time-dependent analog signals produced by typical videocameras. Digitization means conversion to a stream of numbers — preferably *integers* for efficiency.

Since the graph in Figure 6.1 is two-dimensional, to fully digitize the signal shown we have to *sample* in each dimension — in time and in amplitude. Sampling means measuring the quantity we are interested in, usually at evenly spaced intervals. The first kind of sampling — using measurements only at evenly spaced time intervals — is simply called *sampling* (surprisingly), and the rate at which it is performed is called the *sampling frequency*. Figure 6.2(a) shows this type of digitization.

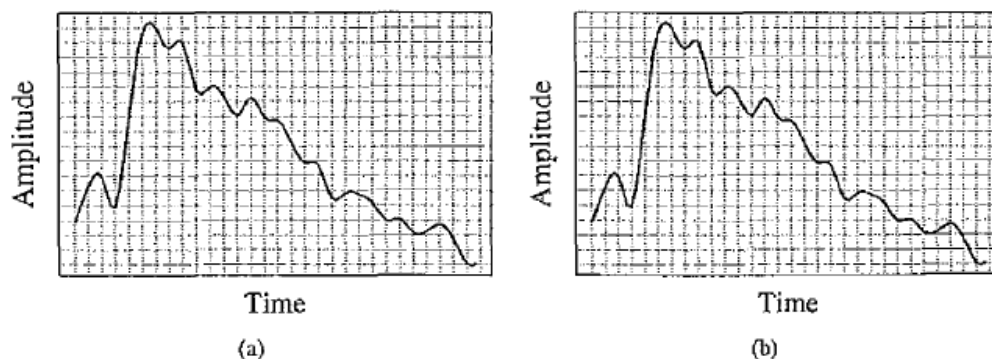


FIGURE 6.2: Sampling and quantization: (a) sampling the analog signal in the time dimension; (b) quantization is sampling the analog signal in the amplitude dimension.

Aliasing occurs when the sampling rate is too low, making the signal appear different from its actual form.

Why does aliasing happen?

- If we sample too slowly, high-frequency signals **blend** and appear as lower frequencies. This creates a **false frequency** (alias) that **wasn't in the original signal**.

How to Prevent Aliasing?

- **Use a Higher Sampling Rate** → Follow Nyquist's rule (**sample at least twice the highest frequency**).
- **Apply an Anti-Aliasing Filter** → Remove high-frequency components **before sampling** to avoid false signals.

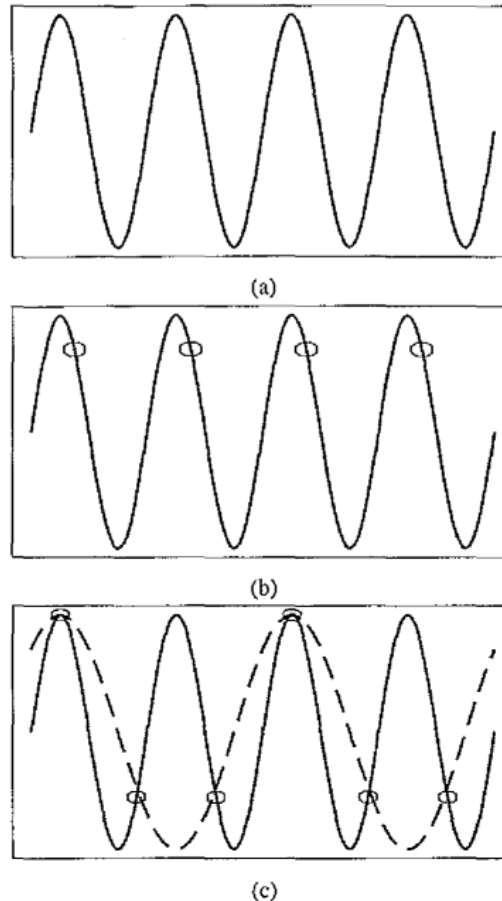


FIGURE 6.4: Aliasing: (a) a single frequency; (b) sampling at exactly the frequency produces a constant; (c) sampling at 1.5 times per cycle produces an *alias* frequency that is perceived.

Now if the sampling rate just equals the actual frequency, we can see from Figure 6.4(b) that a false signal is detected: it is simply a constant, with zero frequency. If, on the other hand, we sample at 1.5 times the frequency, Figure 6.4(c) shows that we obtain an incorrect (*alias*) frequency that is lower than the correct one — it is half the correct one (the wavelength, from peak to peak, is double that of the actual signal). In computer graphics, much effort is aimed at masking such alias effects by various methods of antialiasing. An alias is any artifact that does not belong to the original signal. Thus, for correct sampling we must use a sampling rate equal to at least twice the maximum frequency content in the signal. This is called the *Nyquist rate*.

Nyquist Theorem:

Nyquist Theorem states that to accurately sample a signal **without losing any information**, the sampling rate must be at least **twice the highest frequency** in the signal.

Why is this important?

When we convert an analog signal (like sound or an image) into digital data, we take "snapshots" of it at regular intervals (sampling). If we don't take enough samples, we might misinterpret the original signal, leading to distortion.

Example:

- If a signal contains frequencies up to **10 kHz**, we must sample at **at least 20 kHz** to capture it correctly.
- This required minimum sampling rate is called the **Nyquist rate** ($2 \times$ highest frequency).

Signal-to-Noise Ratio (SNR)

In any analog system, random fluctuations produce *noise* added to the signal, and the measured voltage is thus incorrect. The ratio of the power of the correct signal to the noise is called the *signal-to-noise ratio* (SNR). Therefore, the SNR is a measure of the quality of the signal.

The SNR is usually measured in *decibels* (dB), where 1 dB is a tenth of a *bel*. The SNR value, in units of dB, is defined in terms of base-10 logarithms of squared voltages:

$$SNR = 10 \log_{10} \frac{V_{signal}^2}{V_{noise}^2} = 20 \log_{10} \frac{V_{signal}}{V_{noise}} \quad (6.2)$$

The power in a signal is proportional to the square of the voltage. For example, if the signal voltage V_{signal} is 10 times the noise, the SNR is $20 \times \log_{10}(10) = 20$ dB.

Signal-to-Quantization-Noise Ratio (SQNR)

For digital signals, we must take into account the fact that only quantized values are stored. For a digital audio signal, the precision of each sample is determined by the number of bits per sample, typically 8 or 16.

$$\begin{aligned} SQNR &= 20 \log_{10} \frac{V_{signal}}{V_{quan.noise}} = 20 \log_{10} \frac{2^{N-1}}{\frac{1}{2}} \\ &= 20 \times N \times \log 2 = 6.02N(\text{dB}) \end{aligned}$$

Musical Instrument Digital Interface (MIDI)

- It forms a protocol adopted by the electronic music industry that enables computers, synthesizers, keyboards, and other musical devices to communicate with each other. A synthesizer produces synthetic music and is included on sound cards.

MIDI (Musical Instrument Digital Interface) is a communication protocol that allows electronic musical instruments, computers, and other devices to connect and control each other.

How MIDI Works

- Unlike an audio file (which stores actual sound), MIDI only stores **instructions** about music.
- These instructions include:
 - Which note is played (e.g., C, D, E, etc.)
 - When the note starts and stops
 - How loud the note is (velocity)
 - Which instrument sound to use (piano, violin, drums, etc.)
- The actual sound is generated by a synthesizer or software instrument when the MIDI file is played.

Synthesizer

- A **synthesizer** is an electronic device that **creates and modifies sound** by adjusting pitch, loudness, and tone color. Some synthesizers are standalone devices, while others are built into PC sound cards. Devices that generate sound are called **tone modules or sound modules**.

Sequencer

- A **sequencer** is a tool that **records and arranges musical events** in the form of MIDI data. Originally, sequencers were **hardware devices**, but now they are mostly **software-based music editors** on computers. Sequencers help musicians **edit, rearrange, and play back** MIDI notes.

MIDI Keyboard

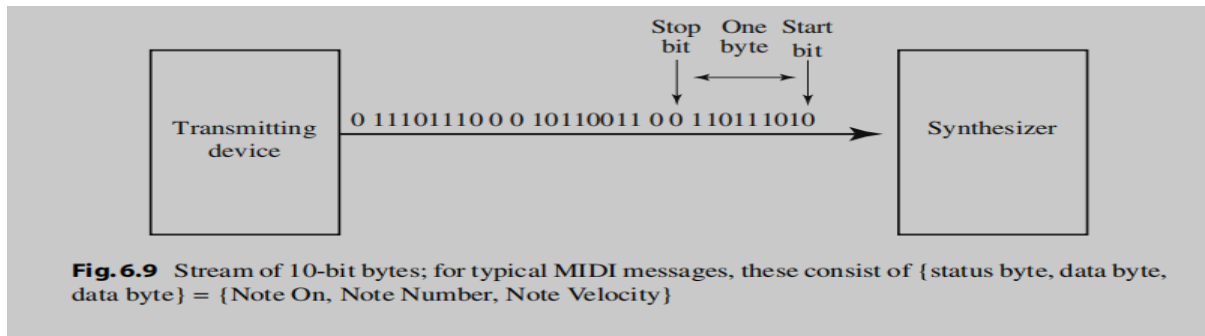
- A **MIDI keyboard** does **not** produce sound by itself. Instead, it generates **MIDI messages** (digital instructions) that tell a synthesizer or computer **which notes to play, how loud, and for how long**.

MIDI concepts

Music is organized into tracks in a sequencer. Each track can be turned on or off on recording or playing back. Usually, a particular instrument is associated with a MIDI channel. MIDI channels are used to separate messages. There are 16 channels, numbered from 0 to 15.

- Along with channel messages (which include a channel number), several other types of messages are sent, such as a general message for all instruments indicating a change in tuning or timing; these are called system messages. It is also possible to send a special message to an instrument's channel that allows sending many notes without a channel specified.

- The way a synthetic musical instrument responds to a MIDI message is usually by simply ignoring any “play sound” message that is not for its channel. If several messages are for its channel, say several simultaneous notes being played on a piano, then the instrument responds, provided it is multi-voice—that is, can play more than a single note at once.

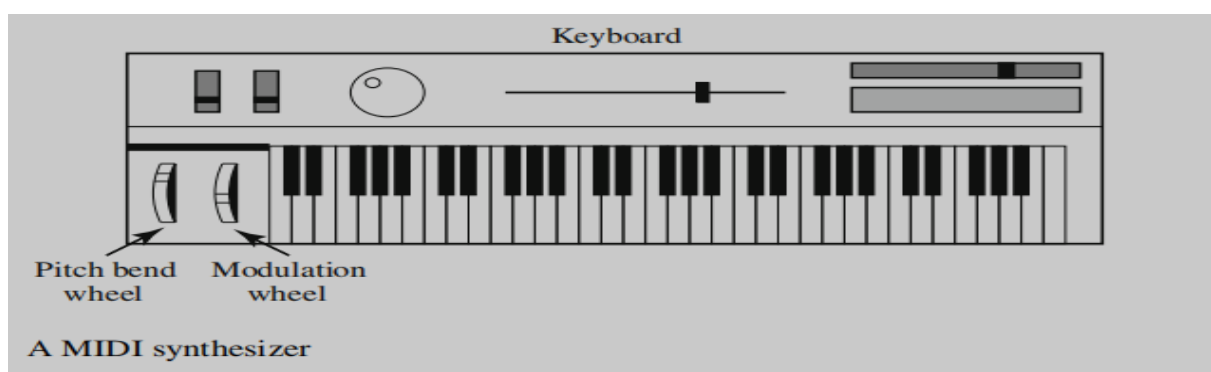


Timbre (pronounced *tam-ber*) is what makes a sound **unique** even when two instruments play the **same note** at the **same pitch** and **loudness**. A piano and a violin can both play the note **C** at **the same loudness** and **the same pitch (frequency)**, but they still sound **different**. This difference in sound is called **timbre**.

The term polyphony refers to the number of voices that can be produced at the same time. So a typical tone module may be able to produce “64 voices of polyphony” (64 different notes at once) and be “16-part multi-timbral” (can produce sounds like 16 different instruments at once).

Hardware aspects of MIDI

- The MIDI hardware setup consists of a 31.25 kbps (kilobits per second) serial connection, with the 10-bit bytes including a 0 start and stop bit.

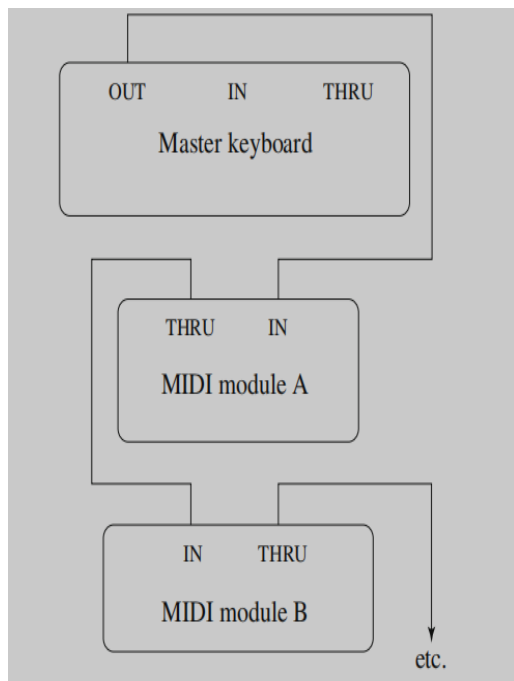


- The modulation wheel adds vibrato. Pitch bend alters the frequency, much like pulling a guitar string over slightly. There are often other controls, such as foot pedals, sliders.
- The physical MIDI ports consist of 5-pin connectors labeled IN and OUT and there can also be a third connector, THRU.

- This last data channel simply copies data entering the IN channel.
- MIDI communication is half-duplex.
- MIDI IN is the connector via which the device receives all MIDI data.
- MIDI OUT is the connector through which the device transmits all the MIDI data it generates itself.
- MIDI THRU is the connector by which the device echoes the data it receives from MIDI IN (and only that—all the data generated by the device itself is sent via MIDI OUT).

MIDI OUT of the keyboard is connected to the MIDI IN of a synthesizer and then THRU to each of the additional sound modules.

- During recording, a keyboard-equipped synthesizer sends MIDI messages to a sequencer, which records them.
- During playback, messages are sent from the sequencer to all the sound modules and the synthesizer, which play the music.



MIDI Message Transmission

- MIDI (Musical Instrument Digital Interface) provides a standardized way for electronic musical instruments, computers, and other audio devices to communicate.
- MIDI messages contain digital data that convey performance information, like the note being played, its duration, and velocity (how hard the note is pressed).
- These messages are transmitted between devices over a variety of physical connections (e.g., MIDI DIN cables, USB, or wireless protocols).
- The process of MIDI message transmission is essential for ensuring proper communication and synchronization across different devices in a MIDI network.

- MIDI messages are transmitted, including the components involved and the types of messages that are sent.

Components Involved in MIDI Message Transmission

- MIDI messages are typically transmitted from one device to another using a point-to-point or daisy-chained connection. The key components involved in the transmission process are:

1. MIDI Controller (Source Device):

- A MIDI controller (e.g., a keyboard, drum pad, or wind controller) sends MIDI messages when the user interacts with the device. The controller generates MIDI events based on physical input (such as pressing a key or hitting a drum pad).

2. MIDI Cable or Interface:

- MIDI DIN Cable: This traditional 5-pin cable is used for connecting MIDI devices in most setups.
- USB: Many modern MIDI devices use USB for easier integration with computers and mobile devices.
- Wireless MIDI: Technologies like Bluetooth MIDI allow devices to communicate without physical cables.

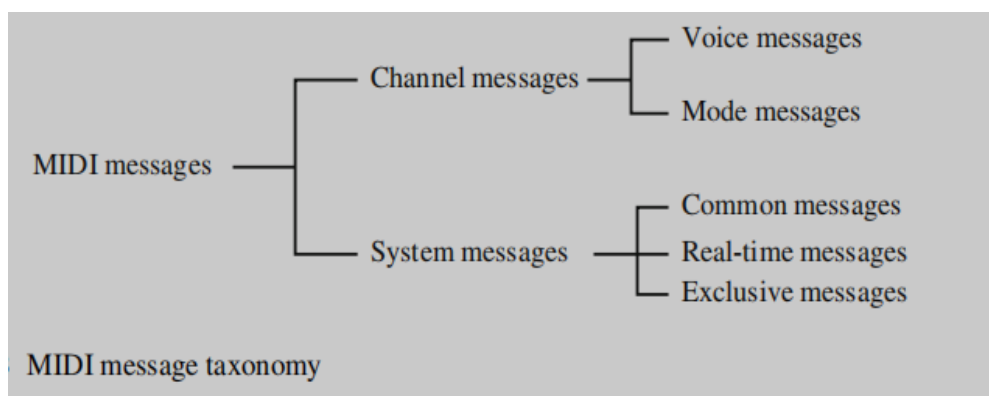
3. MIDI Receiver (Destination Device):

- A MIDI receiver (e.g., a synthesizer, sound module, or DAW) receives MIDI messages and interprets the data to perform actions, such as playing a note, changing a parameter, or triggering an event.

4. MIDI Thru Ports (Optional):

- Some devices include a MIDI Thru port that allows the MIDI signal to pass through to other devices, forming a daisy chain of interconnected MIDI devices.

STRUCTURE OF MIDI MESSAGES



•

- MIDI messages consist of a series of bytes that encode musical data. These messages can be classified into two types: Channel Messages and System Messages.

1. Channel Messages

- Channel messages are the most common type of MIDI message, and they control the performance of an instrument or synthesizer. A channel message can have up to 3 bytes; the first is the status byte (the opcode, as it were), out of this 8 bit, its most significant bit set to 1. next four low order bits identify which of the 16 possible channels this message belongs to, with the three remaining bits holding the message.
- For a data byte, the most significant bit is set to zero.
- Data Bytes: One or two additional bytes that carry specific parameters (like the pitch, velocity, or control changes).
- **Voice Messages:** This type of channel message controls a voice—that is, sends information specifying which note to play or to turn off—and encodes key pressure. Voice messages are also used to specify controller effects, such as sustain, vibrato, tremolo, and the pitch wheel.

TABLE 6.3: MIDI voice messages

Voice message	Status byte	Data byte1	Data byte2
Note Off	&H8n	Key number	Note Off velocity
Note On	&H9n	Key number	Note On velocity
Polyphonic Key Pressure	&HAN	Key number	Amount
Control Change	&HBn	Controller number	Controller value
Program Change	&HCn	Program number	None
Channel Pressure	&HDn	Pressure value	None
Pitch Bend	&HEN	MSB	LSB

&H indicates hexadecimal, and *n* in the Status byte hex value stands for a channel number. All values are in 0.. 127 except Controller number, which is in 0.. 120.

- **Channel Mode Messages:** Channel mode messages form a special case of the Control Change message, and therefore all mode messages have opcode B (so the message is "&HBn," or 1011nnnn). However, a Channel Mode message has its first data byte in 121 through 127 (&H79–7F).

TABLE 6.4: MIDI mode messages

1st data byte	Description	Meaning of 2nd data byte
&H79	Reset all controllers	None; set to 0
&H7A	Local control	0 = off; 127 = on
&H7B	All notes off	None; set to 0
&H7C	Omni mode off	None; set to 0
&H7D	Omni mode on	None; set to 0
&H7E	Mono mode on (Poly mode off)	Controller number
&H7F	Poly mode on (Mono mode off)	None; set to 0

2. System Messages

- **System Messages.** System messages have no channel number and are meant for commands that are not channel-specific, such as timing signals for synchronization, positioning information in MIDI sequences, and detailed setup information for the destination device.
- **System Common Messages:-** Table 6.5 sets out these messages, which relate to timing or positioning. Song Position is measured in beats. The messages determine what is to be played upon receipt of a “start” real-time message.
- **System Real-Time Messages:-** Table 6.6 sets out system real-time messages, which are related to synchronization.
- **System Exclusive Message:-** the final type of system message, System Exclusive

messages, is included so that manufacturers can extend the MIDI standard

TABLE 6.5: MIDI System Common messages

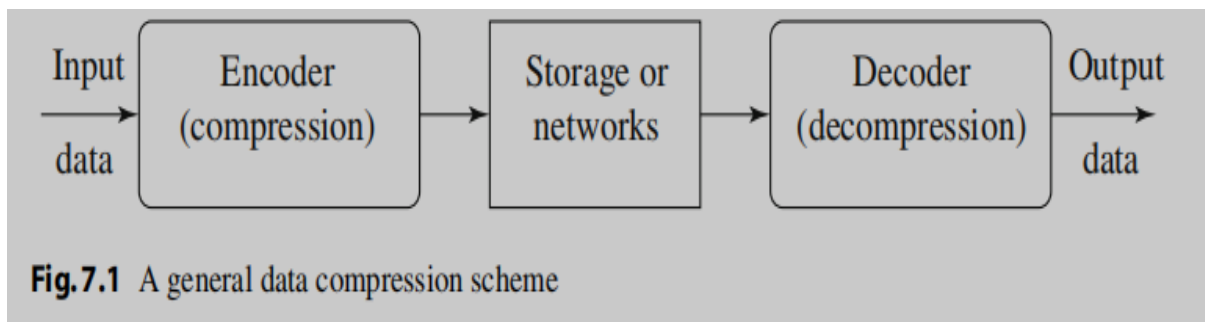
System common message	Status byte	Number of data bytes
MIDI Timing Code	&HF1	1
Song Position Pointer	&HF2	2
Song Select	&HF3	1
Tune Request	&HF6	None
EOX (terminator)	&HF7	None

TABLE 6.6: MIDI System Real-Time messages

System real-time message	Status byte
Timing Clock	&HF8
Start Sequence	&HFA
Continue Sequence	&HFB
Stop Sequence	&HFC
Active Sensing	&HFE
System Reset	&HFF

Lossless Compression Algorithms

Compression is the process of reducing the size of data while maintaining its essential information. It helps save storage space and transmission time while keeping the original data intact or nearly the same.



- compression is performed by an encoder and decompression is performed by a decoder.
- If the compression and decompression processes induce no information loss, the compression scheme is lossless; otherwise, it is lossy.

7.2 Basics of Information Theory

According to the famous scientist Shannon, of Bell Labs [3, 4], the *entropy* η of an information *source* with alphabet $S = \{s_1, s_2, \dots, s_n\}$ is defined as:

$$\eta = H(S) = \sum_{i=1}^n p_i \log_2 \frac{1}{p_i} \quad (7.2)$$

$$= - \sum_{i=1}^n p_i \log_2 p_i \quad (7.3)$$

where p_i is the probability that symbol s_i in S will occur.

The term $\log_2 \frac{1}{p_i}$ indicates the amount of information (the so-called *self-information* defined by Shannon [3]) contained in s_i , which corresponds to the number of bits¹ needed to encode s_i . For example, if the probability of having the character n in a manuscript is $1/32$, the amount of information associated with receiving this character is 5 bits. In other words, a character string nnn will require 15 bits to code.

➤ Run-Length Coding (RLC)

Run-Length Coding (RLC) is a simple **lossless compression** technique used to reduce file size by **replacing repeated data with a shorter representation**. It is most effective for data with **long sequences of repeated values**.

How RLC Works?

- Instead of storing **each repeated value separately**, RLC stores:
 1. **The value itself**
 2. **The number of times it repeats**

Example: Consider the following sequence:

AAAAAABBBBCCCCDDDDDDDD

- Instead of storing every character, RLC stores it as:
(A,6) (B,4) (C,4) (D,8)

➤ Variable-Length Coding

- Shannon–Fano algorithm
- Huffman coding and
- Adaptive Huffman coding.

Shannon–Fano Algorithm

- The Shannon–Fano algorithm was independently developed by Shannon at Bell Labs and Robert Fano at MIT.
- The encoding steps of the Shannon–Fano algorithm can be presented in the following top-down manner:

Symbol	H	E	L	O
Count	1	1	2	1

The encoding steps of the Shannon–Fano algorithm can be presented in the following *top-down* manner:

1. Sort the symbols according to the frequency count of their occurrences.
2. Recursively divide the symbols into two parts, each with approximately the same number of counts, until all parts contain only one symbol.

A natural way of implementing the above procedure is to build a binary tree. As a convention, let us assign bit 0 to its left branches and 1 to the right branches.

- Initially, the symbols are sorted as LHEO. As Fig. 7.3 shows, the first division yields two parts: L with a count of 2, denoted as L:(2); and H, E and O with a total count of 3, denoted as H, E, O:(3). The second division yields H:(1) and E, O:(2). The last division is E:(1) and O:(1).

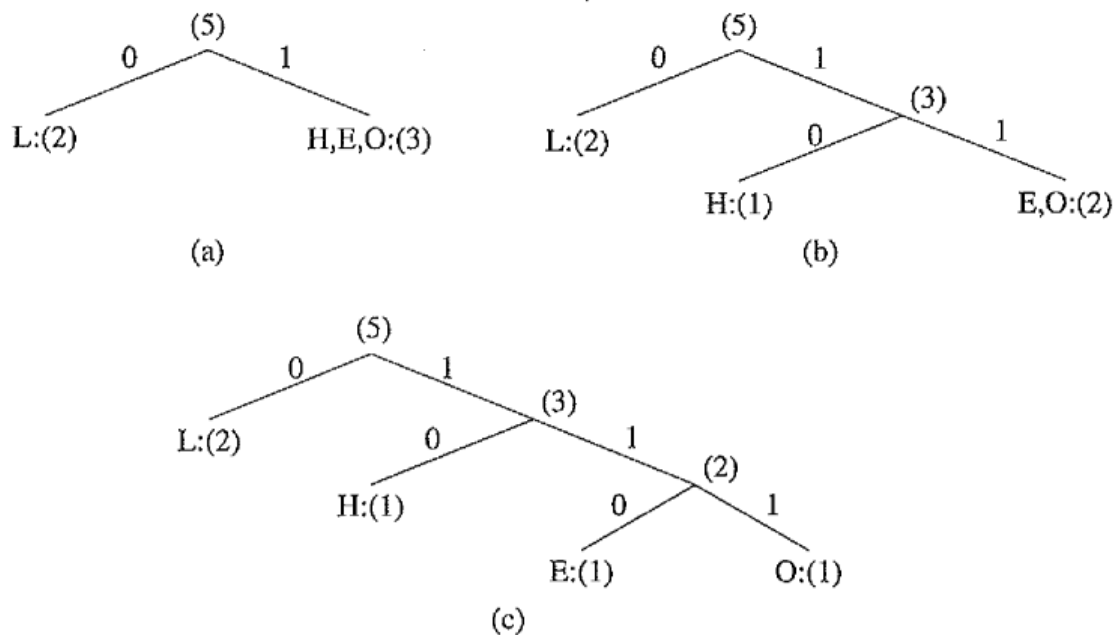


FIGURE 7.3: Coding tree for HELLO by the Shannon–Fano algorithm.

$$\begin{aligned}
\eta &= p_L \cdot \log_2 \frac{1}{p_L} + p_H \cdot \log_2 \frac{1}{p_H} + p_E \cdot \log_2 \frac{1}{p_E} + p_O \cdot \log_2 \frac{1}{p_O} \\
&= 0.4 \times 1.32 + 0.2 \times 2.32 + 0.2 \times 2.32 + 0.2 \times 2.32 = 1.92
\end{aligned}$$

TABLE 7.1: One result of performing the Shannon–Fano algorithm on HELLO.

Symbol	Count	$\log_2 \frac{1}{p_i}$	Code	Number of bits used
L	2	1.32	0	2
H	1	2.32	10	2
E	1	2.32	110	3
O	1	2.32	111	3
TOTAL number of bits:				10

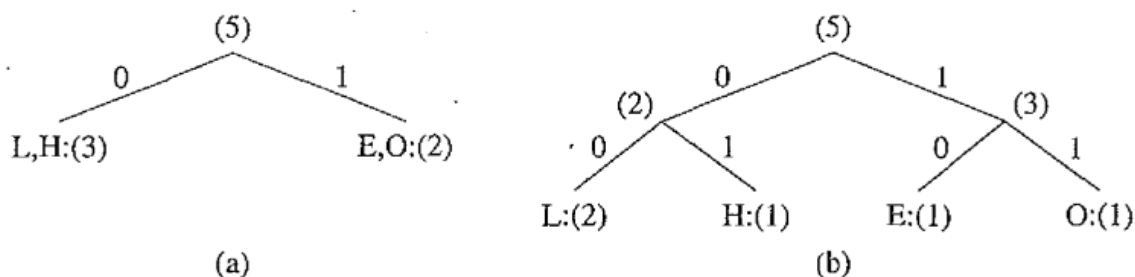


FIGURE 7.4: Another coding tree for HELLO by the Shannon–Fano algorithm.

This suggests that the minimum average number of bits to code each character in the word HELLO would be at least 1.92. In this example, the Shannon–Fano algorithm uses an average of $10/5 = 2$ bits to code each symbol, which is fairly close to the lower bound of 1.92. Apparently, the result is satisfactory.

It should be pointed out that the outcome of the Shannon–Fano algorithm is not necessarily unique. For instance, at the first division in the above example, it would be equally valid to divide into the two parts L,H:(3) and E,O:(2). This would result in the coding in Figure 7.4. Table 7.2 shows the codewords are different now. Also, these two sets of codewords may behave differently when errors are present. Coincidentally, the total number of bits required to encode the word HELLO remains at 10.

Huffman Coding

- Huffman coding is a popular algorithm used in data compression. It helps reduce the size of data by encoding information using variable-length codes, where frequently occurring symbols (characters or data items) are assigned shorter codes, and less frequent symbols are assigned longer codes.
- This technique is widely used in applications like file compression, image compression (e.g., JPEG), and even in transmission protocols.

Advantages of Huffman Coding

1. **Efficient Compression:** Huffman coding can significantly reduce the size of the data, especially when there are large disparities in the frequency of symbols.
2. **Optimal for Prefix-Free Codes:** It guarantees the most efficient compression for any set of symbol frequencies.
3. **Lossless Compression:** Huffman coding is a lossless compression algorithm, meaning the original data can be perfectly reconstructed from the compressed data.
4. **Widely Used:** Huffman coding is used in various applications such as ZIP file compression, JPEG image compression, and even in the MP3 audio format.

•

The following are important properties of Huffman coding:

- **Unique prefix property.** No Huffman code is a prefix of any other Huffman code. For instance, the code 0 assigned to L in Figure 7.5(c) is not a prefix of the code 10 for H or 110 for E or 111 for O; nor is the code 10 for H a prefix of the code 110 for E or 111 for O. It turns out that the unique prefix property is guaranteed by the above Huffman algorithm, since it always places all input symbols at the leaf nodes of the Huffman tree. The Huffman code is one of the *prefix codes* for which the unique prefix property holds. The code generated by the Shannon–Fano algorithm is another such example.

This property is essential and also makes for an efficient decoder, since it precludes any ambiguity in decoding. In the above example, if a bit 0 is received, the decoder can immediately produce a symbol L without waiting for any more bits to be transmitted.

•

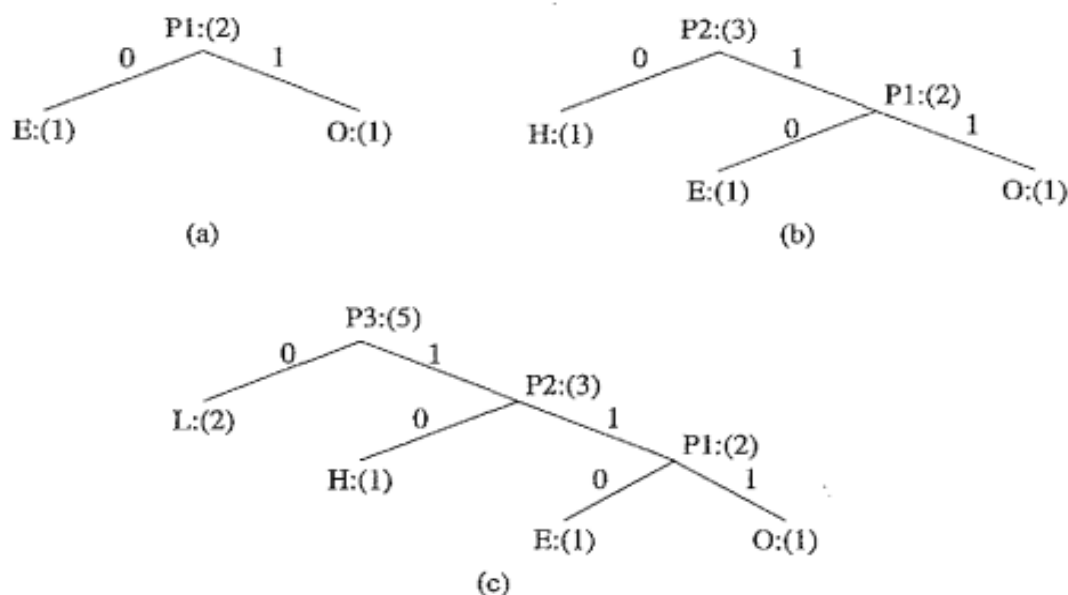


FIGURE 7.5: Coding tree for HELLO using the Huffman algorithm.

ALGORITHM 7.1 HUFFMAN CODING

1. Initialization: put all symbols on the list sorted according to their frequency counts.
 2. Repeat until the list has only one symbol left.
 - (a) From the list, pick two symbols with the lowest frequency counts. Form a Huffman subtree that has these two symbols as child nodes and create a parent node for them.
 - (b) Assign the sum of the children's frequency counts to the parent and insert it into the list, such that the order is maintained.
 - (c) Delete the children from the list.
 3. Assign a codeword for each leaf based on the path from the root. ↩
-

In the above figure, new symbols P1, P2, P3 are created to refer to the parent nodes in the Huffman coding tree. The contents in the list are illustrated below:

After initialization:	L H E O
After iteration (a):	L P1 H
After iteration (b):	L P2
After iteration (c):	P3

Adaptive Huffman Coding

PROCEDURE 7.1 Procedures for Adaptive Huffman Coding

ENCODER

```
Initial_code();
while not EOF
{
    get(c);
    encode(c);
    update_tree(c);
}
```

DECODER

```
Initial_code();
while not EOF
{
    decode(c);
    output(c);
    update_tree(c);
}
```

- `Initial_code` assigns symbols with some initially agreed-upon codes, without any prior knowledge of the frequency counts for them. For example, some conventional code such as ASCII may be used for coding character symbols.
- `update_tree` is a procedure for constructing an adaptive Huffman tree. It basically does two things: it increments the frequency counts for the symbols (including any new ones), and updates the configuration of the tree.
 - The Huffman tree must always maintain its *sibling property* — that is, all nodes (internal and leaf) are arranged in the order of increasing counts. Nodes are numbered in order from left to right, bottom to top. (See Figure 7.6, in which the first node is 1.A:(1), the second node is 2.B:(1), and so on, where the numbers in parentheses indicates the count.) If the sibling property is about to be violated, a *swap* procedure is invoked to update the tree by rearranging the nodes.
 - When a swap is necessary, the farthest node with count N is swapped with the node whose count has just been increased to $N + 1$. Note that if the node with count N is not a leaf-node — it is the root of a subtree — the entire subtree will go with it during the swap.
- The encoder and decoder must use exactly the same `Initial_code` and `update_tree` routines.

****Refer solved example on Notebook.****

➤ Dictionary based coding

DICTIONARY-BASED CODING

The Lempel-Ziv-Welch (LZW) algorithm employs an adaptive, dictionary-based compression technique. Unlike variable-length coding, in which the lengths of the codewords are different, LZW uses fixed-length codewords to represent variable-length strings of symbols/characters that commonly occur together, such as words in English text.

ALGORITHM 7.2 LZW COMPRESSION

```
BEGIN
  s = next input character;
  while not EOF
    {
      c = next input character;

      if s + c exists in the dictionary
        s = s + c;

    else
      {
        output the code for s;
        add string s + c to the dictionary with a new code;
        s = c;
      }
    }
  output the code for s;
END
```

EXAMPLE 7.2 LZW Compression for String ABABBABCABABBA

Let's start with a very simple dictionary (also referred to as a *string table*), initially containing only three characters, with codes as follows:

code	string
-----	-----
1	A
2	B
3	C

Now if the input string is ABABBABCABABBA, the LZW compression algorithm works as follows:

Now if the input string is ABABBABCABABBA, the LZW compression algorithm works as follows:

s	c	output	code	string

			1	A
			2	B
			3	C

A	B	1	4	AB
B	A	2	5	BA
A	B			
AB	B	4	6	ABB
B	A			
BA	B	5	7	BAB
B	C	2	8	BC
C	A	3	9	CA
A	B			
AB	A	4	10	ABA
A	B			
AB	B			
ABB	A	6	11	ABBA
A	EOF	1		

The output codes are 1 2 4 5 2 3 4 6 1. Instead of 14 characters, only 9 codes need to be sent. If we assume each character or code is transmitted as a byte, that is quite a saving (the compression ratio would be $14/9 = 1.56$). (Remember, the LZW is an adaptive algorithm, in which the encoder and decoder independently build their own string tables. Hence, there is no overhead involving transmitting the string table.)

ALGORITHM 7.3 LZW DECOMPRESSION (SIMPLE VERSION)

```

BEGIN
  s = NIL;
  while not EOF
  {
    k = next input code;
    entry = dictionary entry for k;
    output entry;
    if (s != NIL)
      add string s + entry[0] to dictionary
      with a new code;
    s = entry;
  }
END

```

EXAMPLE 7.3 LZW decompression for string ABABBABCABABBA

Input codes to the decoder are 1 2 4 5 2 3 4 6 1. The initial string table is identical to what is used by the encoder.

The LZW decompression algorithm then works as follows:

s	k	entry/output	code	string

			1	A
			2	B
			3	C

NIL	1	A		
A	2	B	4	AB
B	4	AB	5	BA
AB	5	BA	6	ABB
BA	2	B	7	BAB
B	3	C	8	BC
C	4	AB	9	CA
AB	6	ABB	10	ABA
ABB	1	A	11	ABBA
A	EOF			

Apparently the output string is ABABBABCABABBA — a truly lossless result!

➤ Arithmetic Coding

Arithmetic coding is a lossless compression technique that replaces an entire sequence of symbols (characters, numbers, etc.) with a single decimal number between 0 and 1.

Unlike Huffman coding, which assigns fixed-length codes to symbols, arithmetic coding assigns a fractional value to the entire data, leading to better compression for certain types of data.

A message is represented by a half-open interval $[a, b)$ where a and b are real numbers between 0 and 1. Initially, the interval is $[0, 1)$. When the message becomes longer, the length of the interval shortens, and the number of bits needed to represent the interval increases. Suppose the alphabet is $[A, B, C, D, E, F, \$]$, in which $\$$ is a special symbol used to terminate the message, and the known probability distribution is as shown in Figure 7.8(a).

ALGORITHM 7.5 ARITHMETIC CODING ENCODER

BEGIN

low = 0.0; high = 1.0; range = 1.0;

while (symbol != terminator)

{

get (symbol);

low = low + range * Range_low(symbol);

high = low + range * Range_high(symbol);

range = high - low;

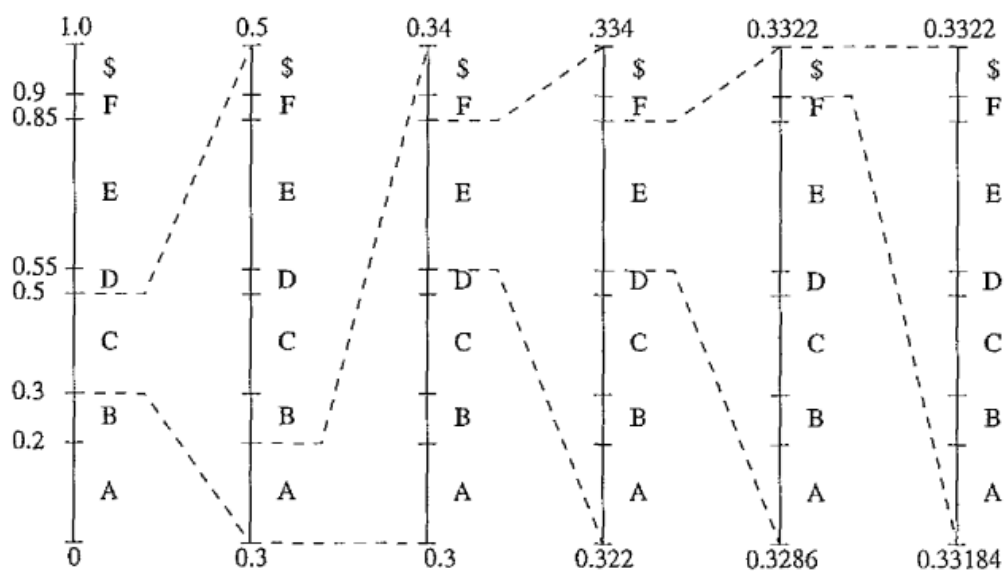
}

output a code so that low ≤ code < high;

END

Symbol	Probability	Range
A	0.2	[0, 0.2)
B	0.1	[0.2, 0.3)
C	0.2	[0.3, 0.5)
D	0.05	[0.5, 0.55)
E	0.3	[0.55, 0.85)
F	0.05	[0.85, 0.9)
\$	0.1	[0.9, 1.0)

(a)



(b)

Symbol	low	high	range
	0	1.0	1.0
C	0.3	0.5	0.2
A	0.30	0.34	0.04
E	0.322	0.334	0.012
E	0.3286	0.3322	0.0036
\$	0.33184	0.33220	0.00036

(c)

For clarity of illustration, the ever-shrinking ranges are enlarged in each step (indicated by dashed lines) in Figure 7.8(b). After the second symbol A, *low*, *high*, and *range* are 0.30, 0.34, and 0.04. The process repeats itself until after the terminating symbol \$ is received. By then *low* and *high* are 0.33184 and 0.33220, respectively. It is apparent that finally we have

$$range = P_C \times P_A \times P_E \times P_E \times P_S = 0.2 \times 0.2 \times 0.3 \times 0.3 \times 0.1 = 0.00036$$

Lossless Image Compression

One of the most commonly used compression techniques in multimedia data compression is differential coding. The basis of data reduction in differential coding is the redundancy in consecutive symbols in a datastream.

1.Differential coding

Differential Coding of Images

Let's consider differential coding in the context of digital images. In a sense, we move from signals with domain in one dimension to signals indexed by numbers in two dimensions (x, y) — the rows and columns of an image. Later, we'll look at video signals. These are even more complex, in that they are indexed by space and time (x, y, t) .

Because of the continuity of the physical world, the gray-level intensities (or color) of background and foreground objects in images tend to change relatively slowly across the image frame. Since we were dealing with signals in the time domain for audio, practitioners generally refer to images as signals in the *spatial domain*. The generally slowly changing

we can define a difference image $d(x, y)$ as follows:

$$d(x, y) = I(x, y) - I(x - 1, y) \quad (7.9)$$

This is a simple approximation of a partial differential operator $\partial/\partial x$ applied to an image defined in terms of integer values of x and y .

Another approach is to use the discrete version of the 2D Laplacian operator to define a difference image $d(x, y)$ as

$$d(x, y) = 4I(x, y) - I(x, y - 1) - I(x, y + 1) - I(x + 1, y) - I(x - 1, y) \quad (7.10)$$

2. Lossless JPEG

Lossless JPEG is a special case of the JPEG image compression. It differs drastically from other JPEG modes in that the algorithm has no lossy steps. Thus we treat it here and consider the more used JPEG methods in Chapter 9. Lossless JPEG is invoked when the user selects a 100% *quality factor* in an image tool. Essentially, lossless JPEG is included in the JPEG compression standard simply for completeness.

The following predictive method is applied on the unprocessed original image (or each color band of the original color image). It essentially involves two steps: forming a differential prediction and encoding.

1. A predictor combines the values of up to three neighboring pixels as the predicted value for the current pixel, indicated by X in Figure 7.10. The predictor can use any one of the seven schemes listed in Table 7.6. If predictor P1 is used, the neighboring intensity value A will be adopted as the predicted intensity of the current pixel; if

predictor P4 is used, the current pixel value is derived from the three neighboring pixels as $A + B - C$; and so on.

2. The encoder compares the prediction with the actual pixel value at position X and encodes the difference using one of the lossless compression techniques we have discussed, such as the Huffman coding scheme.

TABLE 7.6: Predictors for lossless JPEG

Predictor	Prediction
P1	A
P2	B
P3	C
P4	$A + B - C$
P5	$A + (B - C) / 2$
P6	$B + (A - C) / 2$
P7	$(A + B) / 2$